

SESSION - 18

Logging and Debugging



Table of Contents

Table of Contents **1**

Logging and Debugging ***1***

Introduction **1**

Objectives **1**

Log files **2**

Create Custom Log Files. **2**

Debugging in Eclipse **3**

Logging and Debugging

Introduction

Logging and Debugging are two most important area in development. Since we have now seen how we create a website in AEM. We should also see how we can log our logs into log files. Also we should be aware how we can debug our code in Eclipse. Without these things developer can not solve most of the issues. Hence we will talk about these two things in this session.

Objectives

After completing this module, you will be able to:

- Understand what a log file is
- Where log files are created
- We will be creating our own custom log files
- We will open server in debug mode
- We will debug our code in eclipse.

Log files

In AEM you can see log files for your server in a folder named as logs under quick start folder. If you go to this location there will be already some out of the log files located which are

1. request.log
2. error.log
3. access.log
4. audit.log
5. stderr.log
6. stdout.log

As per their names we can understand which file stores logs related to which information. Here most import log file for a developer is error.log file. This file stores all the logs which comes from the code (either from product code or custom code).

Using log files we can also debug our code in higher environments. These log files works as a alarm system for a running project.

Admin teams generally keep track on the log files to keep a check of server's health.

As a developer if there is any error on the page you can always go and check the log files to find the root cause of the problem

Proper logging in a project is very important. It should give sufficient information so that we can figure out the actual issue. Logs should not be misleading.

Create Custom Log Files.

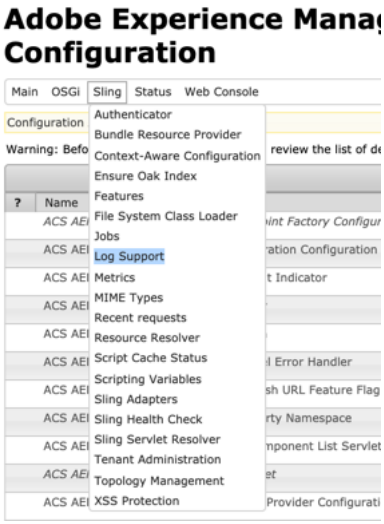
AEM Also provide a way to create custom log files to debug your application. To do that do following

For example if I have to set debug for package my.site.my_site.core.models

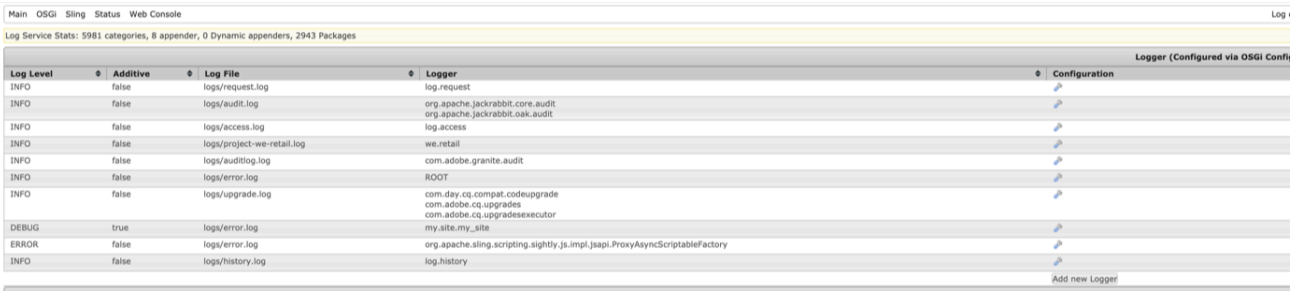
1. Log into Felix Console: **http://<host>:<port>/system/console/configMgr**
2. From "Factory Configurations", create "Apache Sling Logging Writer Configuration"
3. Set value of "Log File" to "../logs/mycustom.log"
4. Click on "Save"
5. From "Factory Configurations", create "Apache Sling Logging Logger Configuration"

6. Set value of "Log Level" to "Debug"
7. Set value of "Log File" to "../logs/mycustom.log"
8. Add "Logger" => my.site.my_site.core.models
9. Click on "Save"

You can also do this from the system console menu Sling->LogSupport as shown in the below image



Go to log support section and then click on **Add new logger** as shown in the image.



Users are often members of Groups, which simplify the allocation of these permissions and/or privileges.

Debugging in Eclipse

To debug an AEM application, you can use java remote debugging.
Warning: Do not enable remote debugging in your production environment unless it necessary as it could greatly affect your application performance.

Step 1: Add the Remote Debugging JVM Parameter

1. To use remote debugging, you must start AEM with this JVM parameter:
   -agentlib:jdwp=transport=dt_socket,address=8000,server=y,suspend=n
2. You can add the parameter by doing any of the following:

3. Add it to your crx-quickstart/bin/start script CQ_JVM_OPTS environment variable (so your server always starts in debug mode).
4. Include it as a parameter when starting AEM with java -jar directly. For example, java **-Xdebug -Xnoagent -Djava.compiler=NONE -Xrunjdwp:transport=dt_socket,server=y,suspend=n,address=8000 -XX:+HeapDumpOnOutOfMemoryError -Xmx1024m -Dorg.apache.sling.commons.log.level=INFO -jar aem-author-p4502.jar -p 4502 -verbose -nofork**

Notes

- If necessary, you can change the port defined under "...,address=8000,..." from 8000 to something that works better in your environment.
- If you notice that it isn't working, avoid forking the java process by specifying the -nofork command line option.
- The start script is located under crx-quickstart/bin

Step 2: Start a remote debugging session

To start a remote debugging session from Eclipse, do the following:

1. Open Eclipse.
2. Choose Run > Debug Configurations.
3. Right-click Remote Java Applications and select New.
4. Select your AEM project under Project.
5. Type in the port from the "address" configuration of the jvm parameter defined above. In the example, it is port 8000 (and the hostname of the machine you are connecting to, most likely localhost)
6. To start a debugging session, double-click your new configuration in the Debug Configurations screen after saving it.

Applies to: Adobe Experience Manager 6.x